





Guidance, Navigation and Control in Drone

BY: AKRITI

INDIA SPACE LAB

Important Announcement!!

Drone being a very vast field, topics taught today is just to make you familiar with various aspect and topics related to drone.

Relax and enjoy today's workshop 😎

Section 1



The World of Autonomy

What are autonomous drone?

- Unmanned Aerial Vehicle (UAV) containing advanced technology
- Operate independently
- Without constant human intervention
- Capability to perform tasks (like dropping delivery parcels)
- Navigate through environment automatically
- Rely on sensors, AI and algorithms



Why Autonomy is Hard?

Uncertain World

1. Sensors are Noisy
2. Delay in Motors
3. External Disturbance (Wind)

Problem Split into 3 parts

1. Navigation (Know State)
2. Guidance (Decide Motion)
3. Control (Make it happen)

Why 3 Parts?

For Modularity

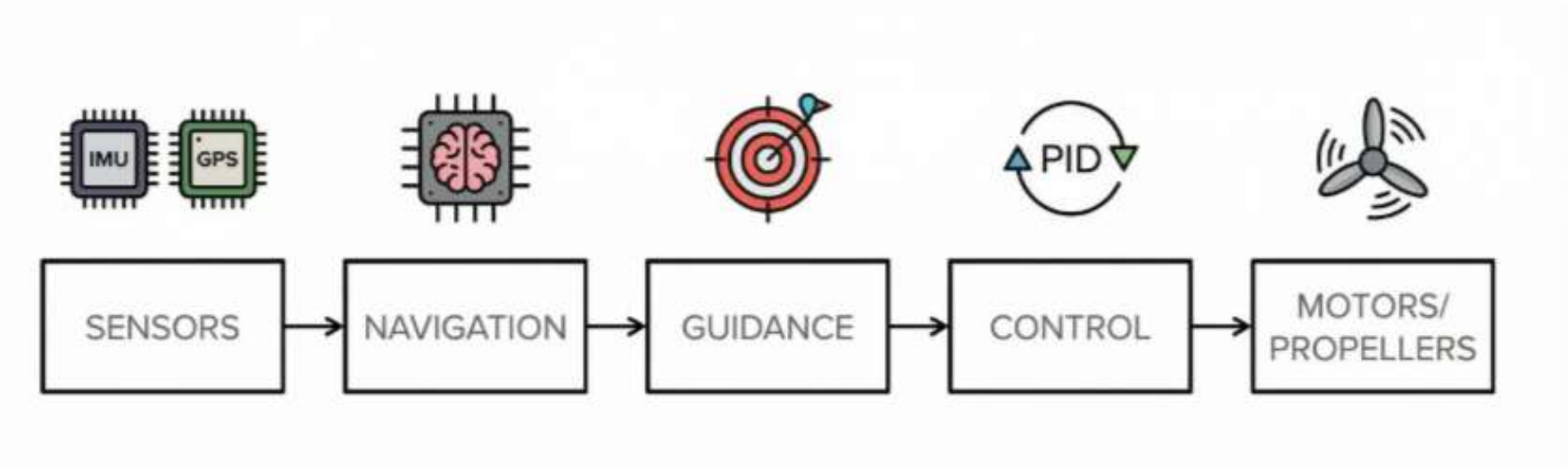
Eg:- Upgrade GPS (Navigation) with changing PID controller (Control)

"Sensors are noisy, the wind blows, and physics doesn't care about your code."

GNC

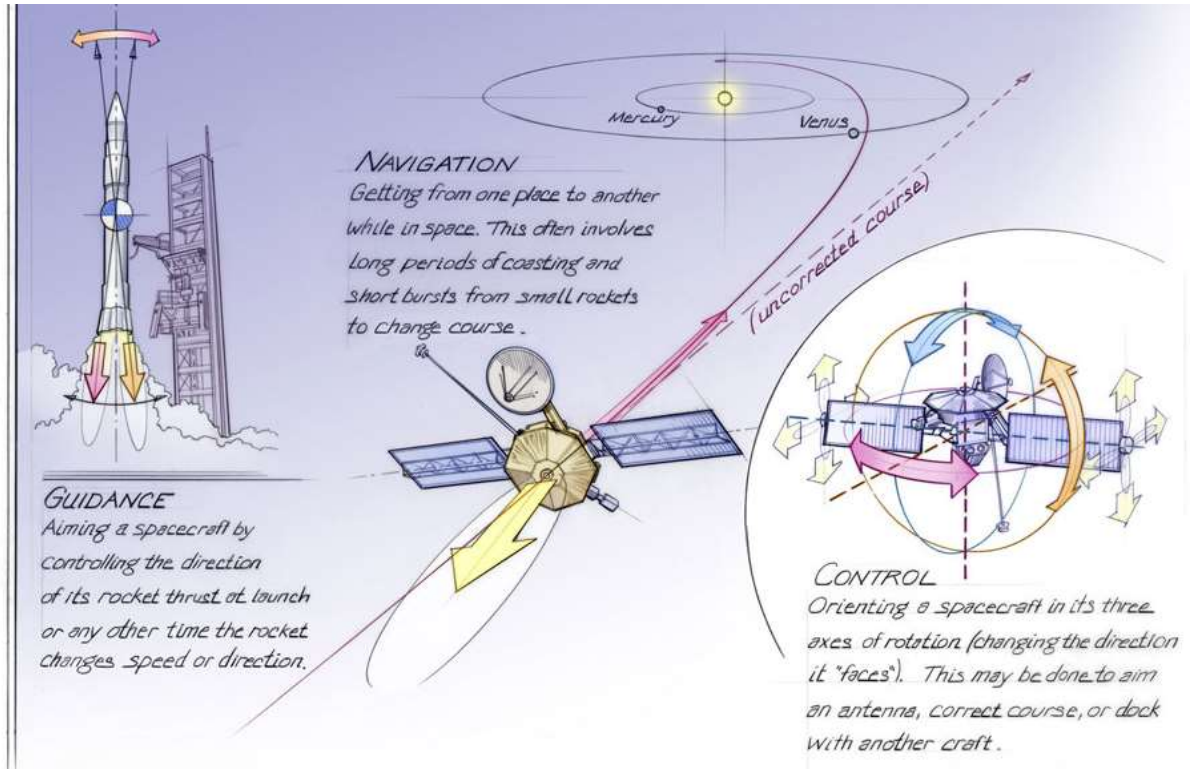


Basic Flow of Autonomy



Navigation tells you where you are, **Guidance** tells you where to go, **Control** makes you move…

Space Mission Analogy



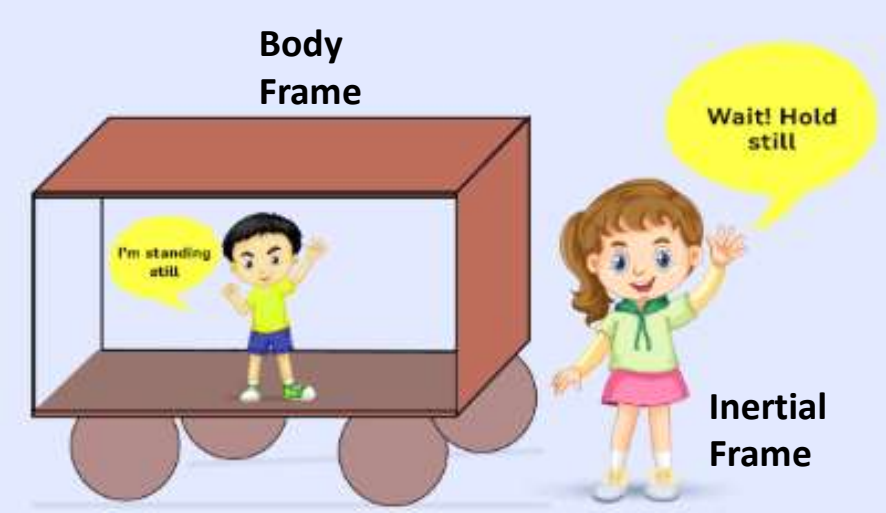
Section 2

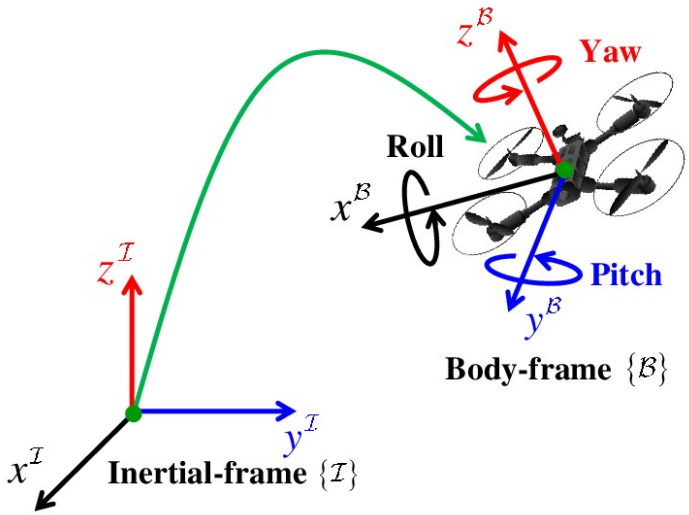


Navigation

How the drone knows its State?

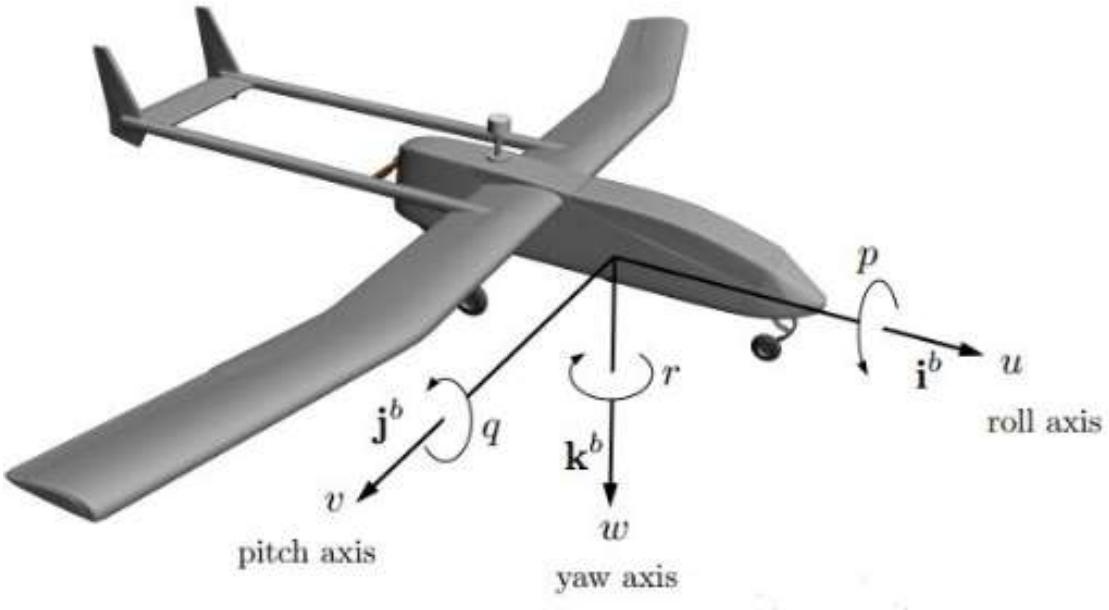
Frame of Reference





What is State?

The 12 numbers that define “flight”

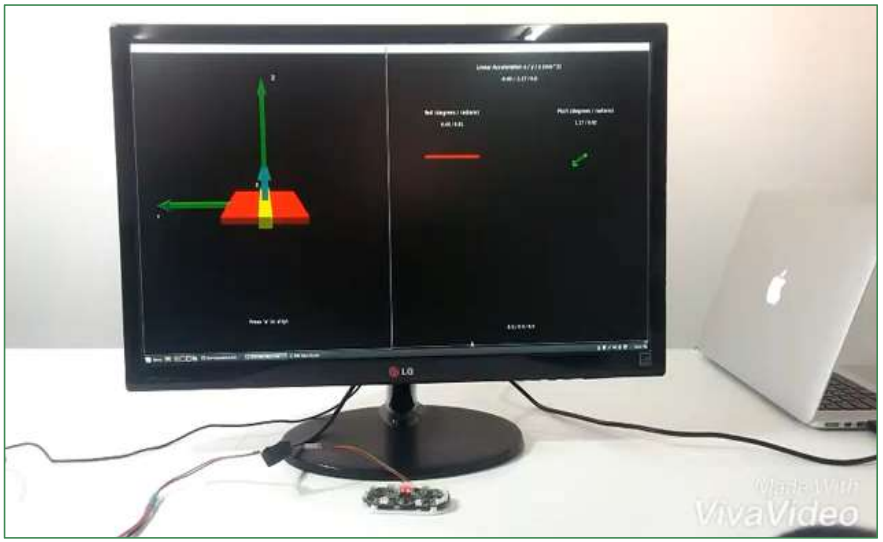


1. **Position (x,y,z)**
   Where is the drone in 3D space?
2. **Velocity (u,v,w)**
   How fast drone is moving in each direction?
3. **Orientation (Roll, Pitch, Yaw)**
   Which way the drone is tilted and facing?
4. **Angular Rates (p,q,r)**
   How fast it is rotating in each direction?

Navigation Sensor: IMU

It Measures:

1. Angular Rate (deg/sec)
2. Acceleration (Specific Force) (g's)



Update Rate:

Extremely fast update rates required for stability

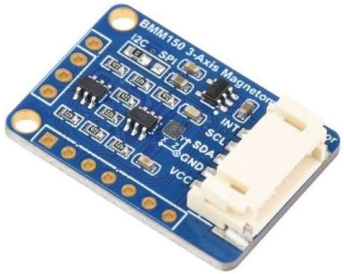
200-1000 Hz

Errors:

- Sensor Bias
- Electrical Noise
- Misalignment

Secondary Sensor

Magnetometer



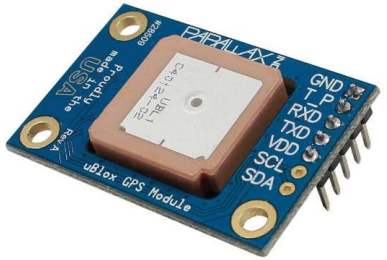
- **Heading (Yaw)**
- **Medium Rate:**
  Updates at 50-100 Hz
- **Weakness:**
  Very sensitive to magnetic interference (metals, wires)

Barometer



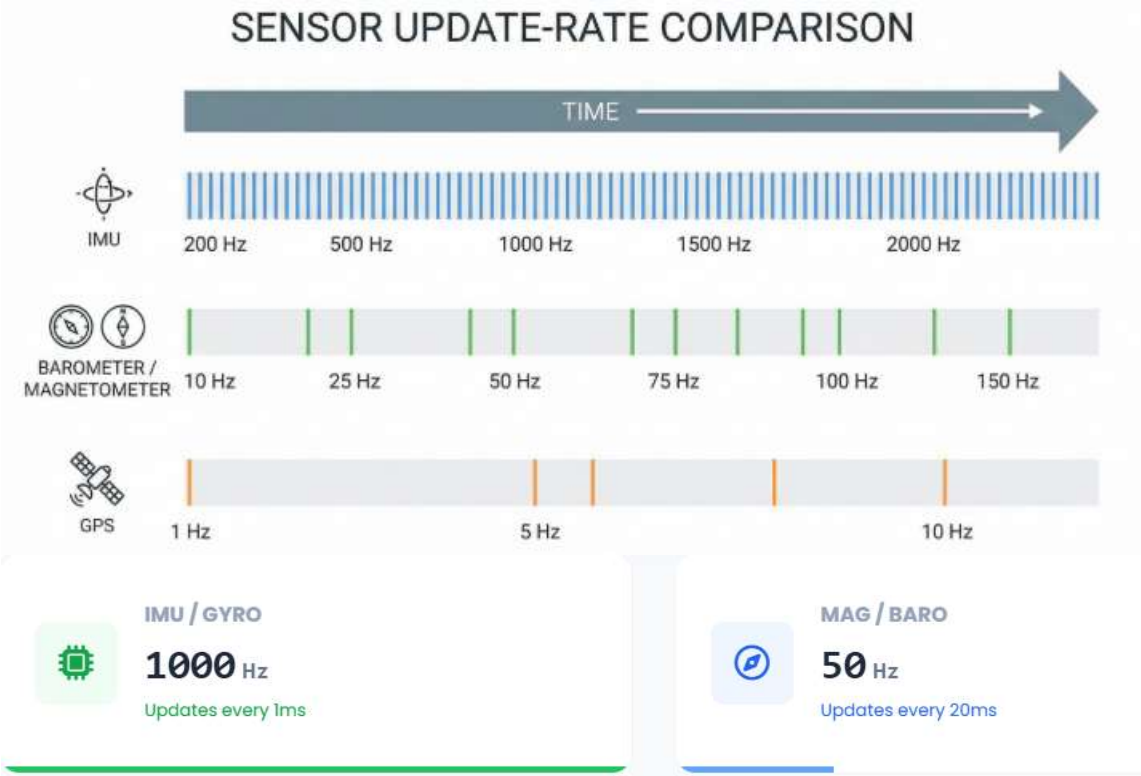
- **Altitude (Z-axis)**
- **Medium Rate:**
  Updates at 10-50 Hz
- **Function:**
  Measures air pressure changes to estimate height

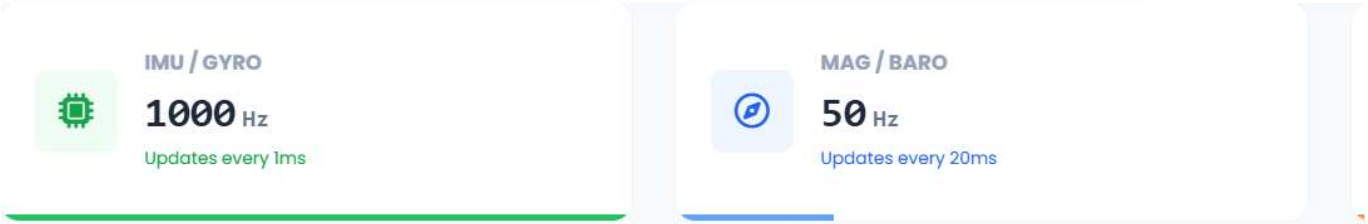
GPS



- **Absolute Position**
- **Medium Rate:**
  Updates at 1-10 Hz
- **Function:**
  Correcting long-term drift (absolute truth)

Sensors Update Rates





The timescale gap between sensors is massive. Can't just rely on GPS..

Why sensors alone aren’t enough?

1. **Sensor Noise**

   Sensors are never perfect. Vibrations from motors create "**jitter**" in accelerometer data.

2. **Integration Drift**

   Integrating velocity to get position accumulates tiny errors over time. Error grows quadratically!

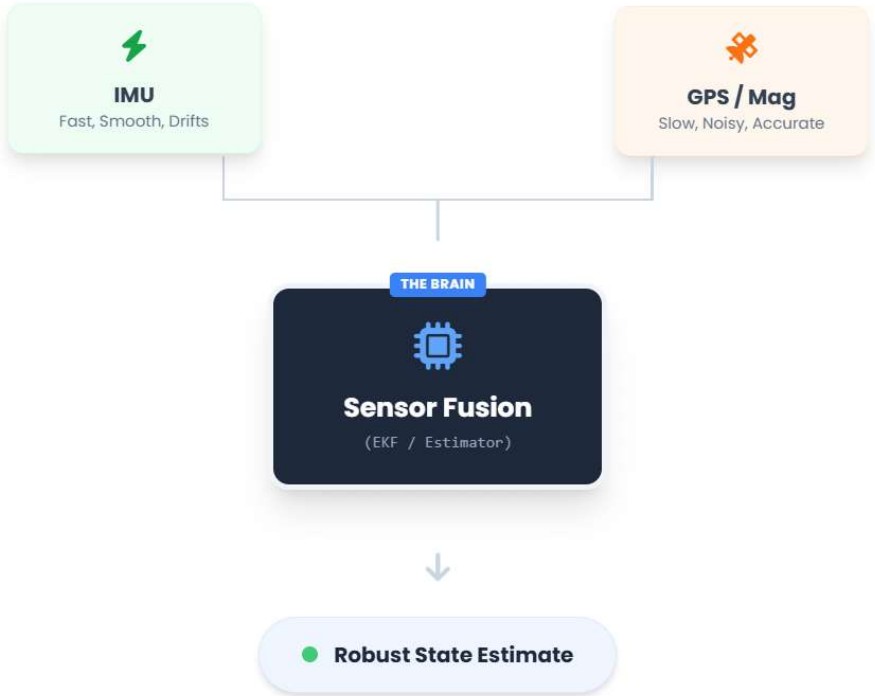
3. **Delay and Bias**

   GPS is slow (lag). Gyros have bias (offset). We need estimation (filtering) to fix this.

Solution: Sensor Fusion

- ✓Merge **Fast sensors** (IMU) for responsiveness with **Slow sensors** (GPS) for accuracy.
- ✓ **Extended Kalman Filter:** A mathematical tool that predicts state and corrects it when new data arrives.

"Navigation is intelligent sensor fusion."



Quiz 1

1. Which sensor provides absolute position? Q1

- A IMU (Gyro/Accel)
- ✓ **GPS / GNSS**
- C Barometer

2. Why do we fuse sensor data? Q2

- A To look cool on diagrams
- ✓ **Reduce noise & correct drift**
- C To save battery power

3. Typical IMU update rate is: Q3

- A 1 Hz (1 per sec)
- B 10 Hz (10 per sec)
- ✓ **200–1000 Hz**

4. True/False: Control acts on estimated state. Q4

- ✓ **TRUE**
- ⊗ FALSE

Control never sees "reality", only the estimate.

Section 3



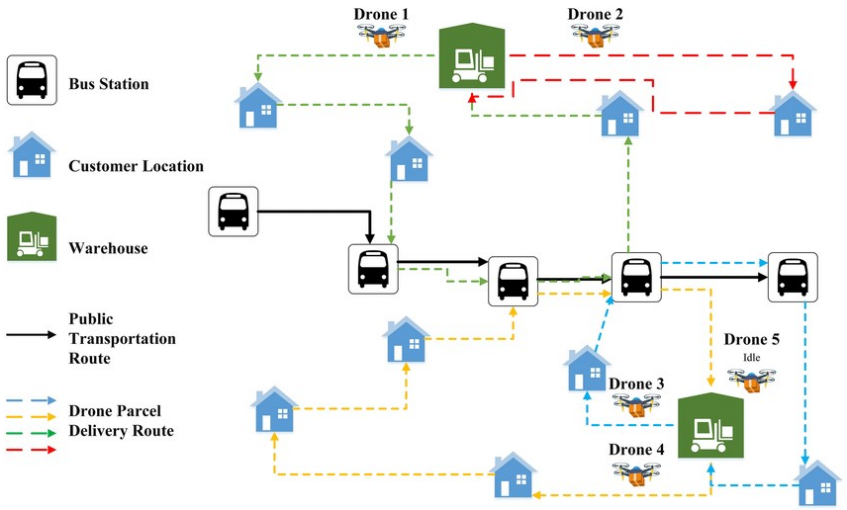
Guidance

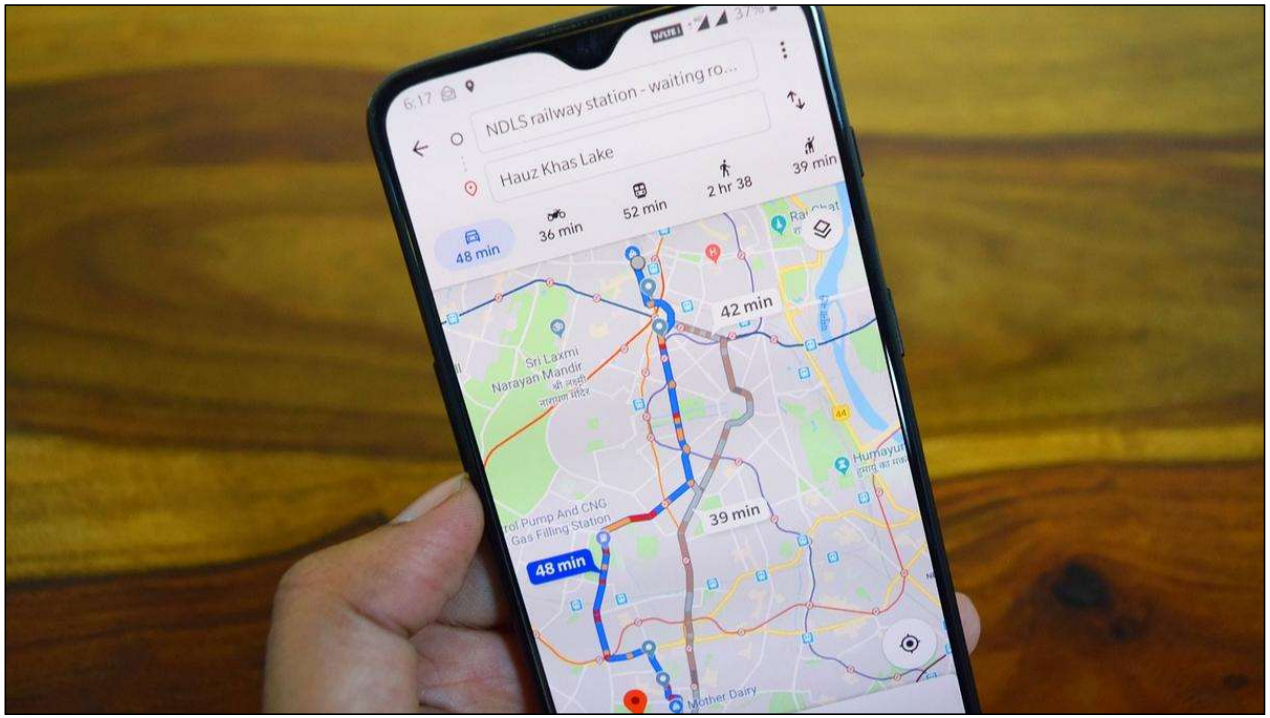
Deciding where to go

What Guidance Does

1. Translates high-level commands like "Loiter here", "Fly to Waypoint B", or "Return to Launch" into mathematical objectives.
2. Generates **target states** for the controller to chase: Target Position, Desired Velocity, and Attitude.

"Guidance is the strategist. It doesn't fly the plane; it decides the path."





Guidance is like choosing a path on Google Map for less time and minimum traffic.

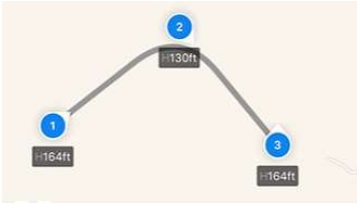
Guidance Commands



Hover at Point

(Position Hold)

Maintains fixed XYZ coordinates. Guidance outputs zero velocity reference.



Fly to Waypoint

(Target Seeking)

Generates velocity vector pointing from current position to target (A → B).



Follow Line

(Path Following)

Minimizes cross-track error to stay on a straight path between two points.

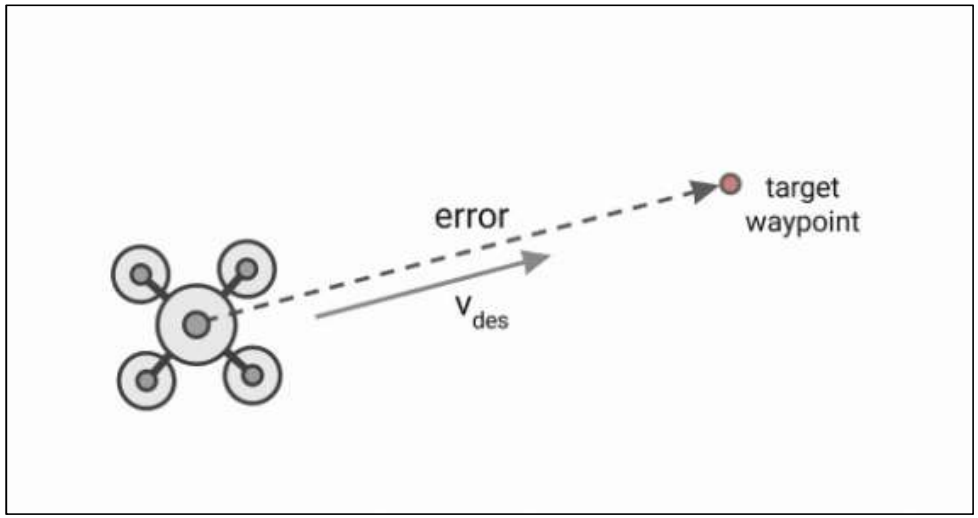


Maintain Alt

(Z-axis Hold)

Adjusts vertical velocity to keep barometer/GPS height constant.

Simple Guidance Logic



The core logic is simple: **Error** = **Target** − **Current**.

The desired velocity vector points directly along this error arrow.

Guidance outputs desired motion, not motor commands!!

Quiz 2

1. Who does Guidance talk directly to? Q1

- A Motors
- ✓ **Controller**
- C Batteries

2. What is the output of Guidance? Q2

- A PWM Signals
- ✓ **References (Pos/Vel/Att)**
- C Raw Gyro Data

3. True/False: Loiter is a guidance task. Q3

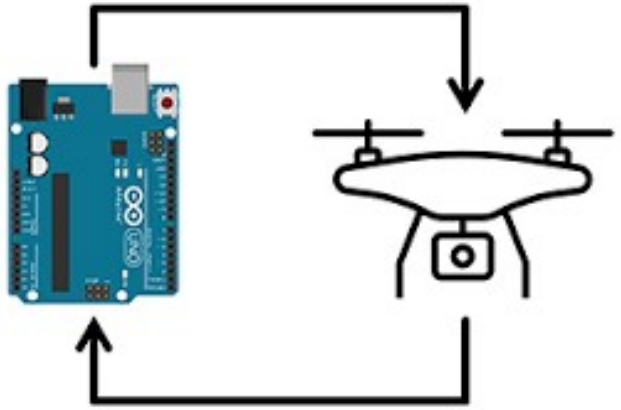
- ✓ **TRUE**
- ⊗ FALSE

Loiter = "stay here" = position reference.

4. The error vector points from: Q4

- A Target → Drone
- ✓ **Drone → Target**
- C Drone → Home

Section 4

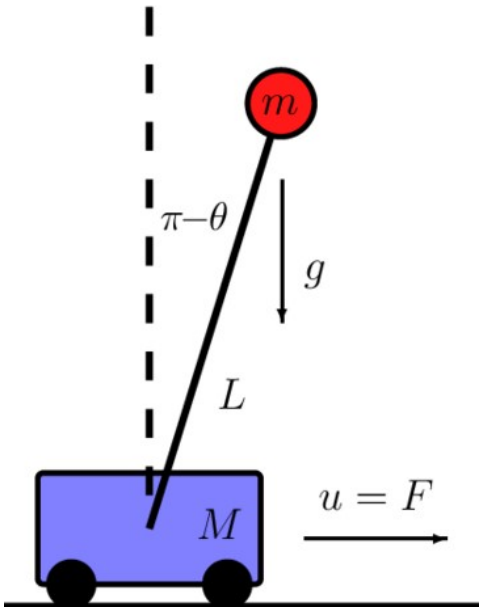


Control

Make drone Obey Physics

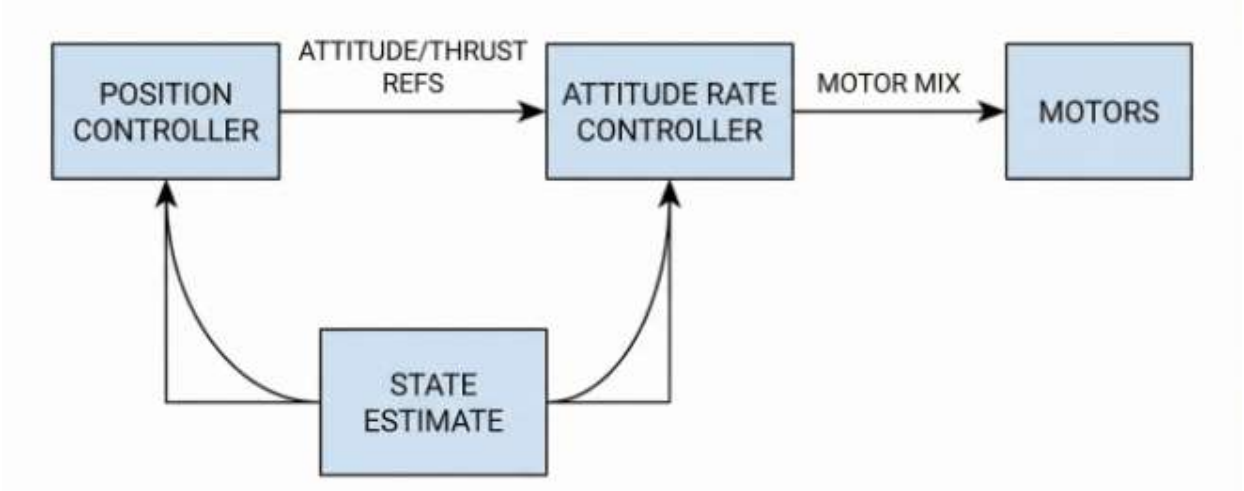
Why Control is Hard

1. A quadcopter is like balancing a broom on your hand. Without active control 400 times a second, it flips instantly.
2. Axes interact. A hard Yaw maneuver can drop your Altitude. Rolling right might induce a slight Pitch.
3. Physics doesn't wait. Sensors must be read and motors updated in milliseconds to catch errors before they grow.



"A drone is naturally unstable — control keeps it alive."

Control Hierarchy



Outer **Position/Velocity** loops cascade references down to inner **Attitude/Rate** loops

PID Control

Proportional (P) - Reaction

- Responds immediately to the **current error**. "If I'm far away, push hard."

Integral (I) – Memory

- Accumulates Error: Looks at the past to fix steady-state drift
- The "Fixer": Pushes the drone back if wind blows it off course

Derivative (D) – Anticipation

- Predicts Future: Reacts to the rate of change to dampen motion
- The "Brake": Prevents overshoot and smooths out the flight



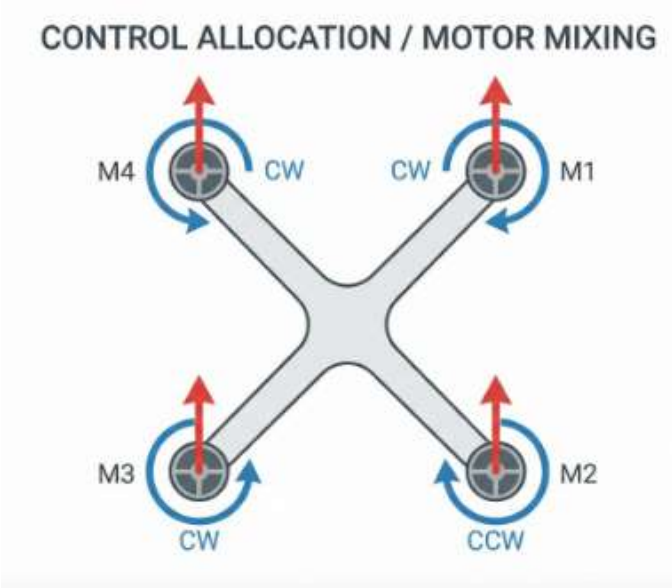
Beginner Approach: Don't rely on complex math; rely on observation

Order:

1. Tune P (Strength)
2. Tune D (Damping)
3. Tune I (Drift)

Tuning Strategy

Control Allocation (Motor Mixing)



Mixing logic distributes **Thrust** to M1-M4, using **CW/CCW** pairs to control Yaw.

Quiz 3

1. The outer control loop typically handles: Q1

- A Attitude (Inner Loop)
- ✓ **Position / Velocity**
- C Motor PWM directly

2. The D-term in PID mainly helps with: Q2

- A System Delay
- B Sensor Noise
- ✓ **Overshoot (Damping)**

3. Which axis uses differential CW/CCW torque? Q3

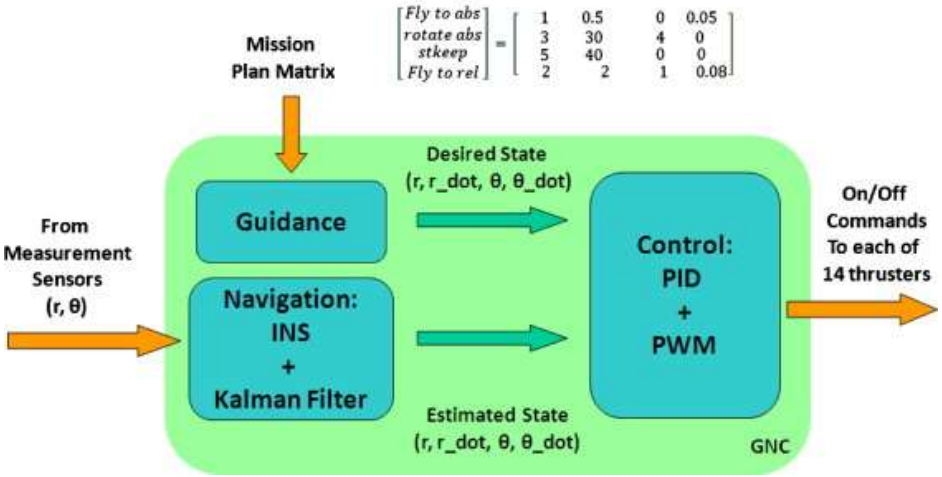
- A Roll Axis
- B Pitch Axis
- ✓ **Yaw Axis**

4. True/False: Higher P always improves stability. Q4

- ⊗ TRUE
- ✓ **FALSE**

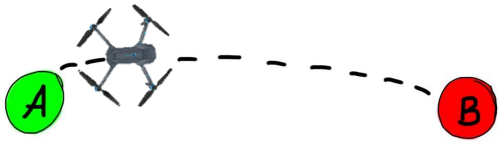
Too much P gain leads to violent oscillations!

Section 5



GNC Integration

Putting GNC Together



Walkthrough Mission

"Fly from Point A to Point B"

This sequence happens hundreds of times per second (e.g., 400Hz) to keep the drone stable and moving towards its goal.

Step 1: Sensors Measure Motion
IMU detects acceleration/ rotation, GPS checks position

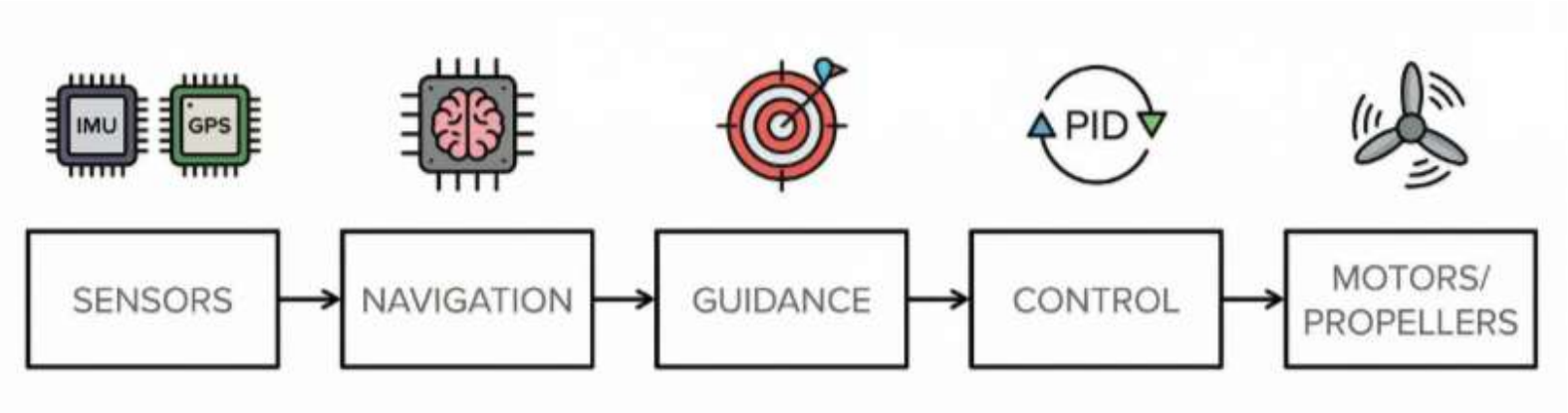
Step 2: Navigation Estimates State
Fuses noisy data to answer: "Where am I?"

Step 3: Guidance Computes References
Calculates error to target & desired velocity

Step 4: Control Generates Commands
PID loops convert errors into motor mixes

Step 5: Motors Produce Forces
Propellers spin → Physics happens → Drone moves

Complete GNC Pipeline



From raw sensor data to spinning propellers…

Note: **Timing** and **latency budgets** are critical at every stage of this loop.

Quiz 4

1. Which role mapping is correct? Q1

- A Nav=Action, Ctrl=State
- ✓ **Nav=State, Guide=Refs, Ctrl=Action**
- C Guide=State, Nav=Refs

2. Which sensor is slow but absolute? Q2

- A IMU (Accelerometer)
- B Gyroscope
- ✓ **GPS / GNSS**

3. What does Guidance send to Control? Q3

- ✓ **Attitude / Velocity References**
- B PWM Signals
- C Battery Current

4. "Navigation is _____ sensor fusion." Q4

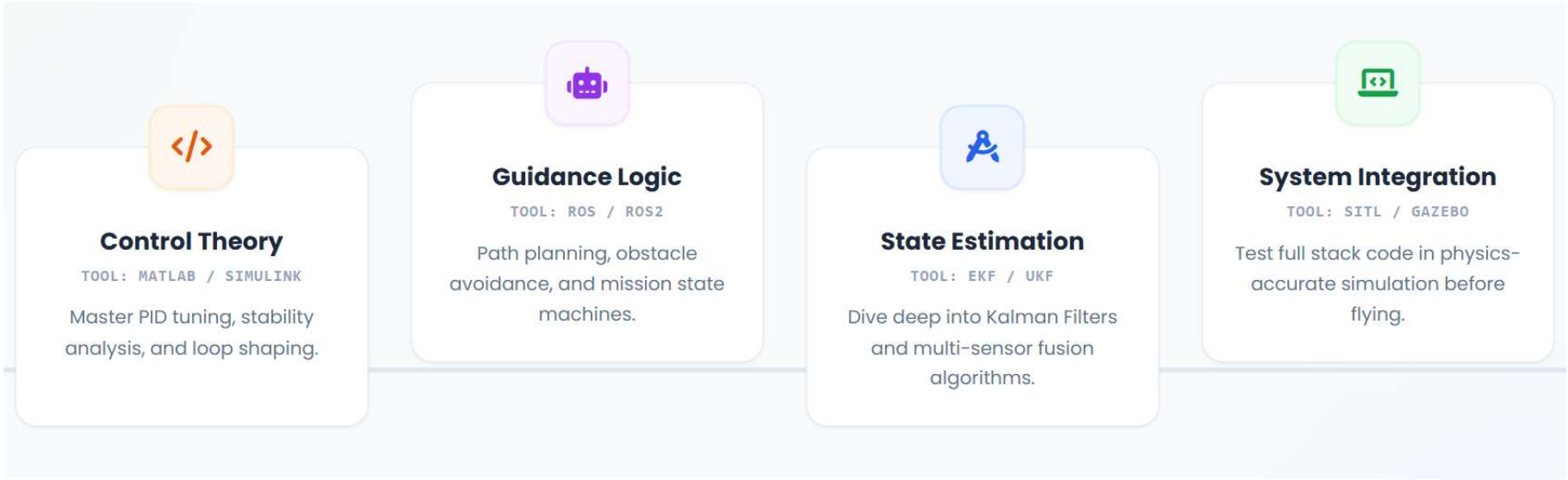
- A "random"
- ✓ **"intelligent"**
- C "expensive"

Section 6



Conclusion

Learning Roadmap



"You now understand the language of drone autonomy."

Keep building. Keep crashing. Keep learning.

Thank You ☺